

Sunrise 2027 — One Identifier Thread, From Factory to Return

A QR on a package is **not** a Sunrise 2027 QR. The gate is the **registry**, not the format — and the same identifier already runs **factory → warehouse → sale → return → recall**, whether or not anyone models it.

For: anyone who owns product data, fulfilment, or returns across more than one market.

Not: a packaging story. The barcode is where the thread becomes visible, not where it starts.

THE MISCONCEPTION, FIRST

A QR code on a package is not a Sunrise 2027 QR code.

For a checkout to scan it, it has to be a **GS1 Digital Link** carrying a valid GTIN — a URI of the form `https://yourdomain.com/01/09506000134352`. A marketing QR that points at a landing page will work fine on a phone and fail completely at the till.

That distinction is most of the confusion in the market right now, and it costs artwork revisions to discover late.

WHAT SUNRISE 2027 ACTUALLY IS

GS1's target is that retailers globally are **able to scan 2D barcodes at point of sale by the end of 2027**. It is a retailer capability date, which makes it a brand readiness date.

Two things it is not:

- **It is not a switch-off.** Linear barcodes keep working. The transition is coexistence, not replacement, and packs will carry both for years.

- **It is not automatic.** Nothing about your existing UPC becomes a Digital Link by itself.

Reference: [GS1 2D barcodes](#) · [GS1 Digital Link](#)

THE NUMBERS, AND WHICH OF THEM IS REAL

Four identifiers get used interchangeably in conversation and are not interchangeable at all.

	ASSIGNED BY	UNIQUE WHERE	PORTABLE	SCANNABLE AT POS
GTIN	GS1 registry	Globally	✓	✓
MPN	You	Within your brand	Partly	×
SKU	You	Your systems only	×	×
ASIN	Amazon	Amazon	×	×

GTIN comes in four widths – GTIN-8, GTIN-12 (UPC-A), GTIN-13 (EAN-13) and GTIN-14 (cases). Fourteen digits is the canonical **field** width: GS1 specifies that all GTINs are right-justified with leading zeros into a 14-digit field for processing, so a GTIN-13 stored correctly is `05901234123457`. There is no 16-digit GTIN. The 18-digit key in the family is the SSCC, for shipping containers.

A GTIN-13 decomposes as:

5 901234 12345 7

| |

|

└─ check digit (1) computed

| |

└─ item reference (5) you assign

|

└─ company prefix (6) GS1 assigns

└─ from your licence

The check digit is arithmetic you can verify yourself — alternate $\times 1$ and $\times 3$ across the first twelve digits, sum, subtract from the next multiple of ten:

$$\begin{aligned} & 5(1) \ 9(3) \ 0(1) \ 1(3) \ 2(1) \ 3(3) \ 4(1) \ 1(3) \ 2(1) \ 3(3) \ 4(1) \ 5(3) \\ & = 5+27+0+3+2+9+4+3+2+9+4+15 = 83 \quad \rightarrow \quad 90 - 83 = 7 \checkmark \end{aligned}$$

MPN is self-assigned, unregistered, has no check digit and no format. It is unique only within your brand, which is why Google requires *brand + MPN together* when a GTIN is absent. **SKU** is narrower still — merchant-internal, never an external identity. **ASIN** is Amazon's, exists inside Amazon, and stops at their boundary.

THE GATE IS THE REGISTRY, NOT THE FORMAT

This is the part teams get wrong, and it is expensive.

A well-formed number is not a valid GTIN. The check digit proves the number is *structurally* correct; only the GS1 registry proves it is **yours**. Amazon validates listed GTINs against GS1's records rather than accepting anything that passes the arithmetic — which is how a large population of sellers discovered that the cheap resold barcodes they had bought resolved to somebody else's company prefix. The listings were suppressed and the remedy was a real licence.

So the first step is commercial, not technical: a GS1 company prefix, from which item references and therefore GTINs are assigned. No membership, no GTIN, no Digital Link. (Get barcodes)

Budget the lead time. It is the longest pole in the project and it has nothing to do with engineering.

Here is the part that makes this operational rather than cosmetic. The same identifiers are already load-bearing across the physical estate, and nobody thinks of them as one thing:

- **MPN → the factory.** The part being built, the BOM line, the firmware revision bound to it. Pick-and-pack runs off it.
- **Barcode → the WMS.** Receiving, putaway, shelf location, pick, pack, ship. Every scan in the building is that number.
- **The same code → fulfilment,** onto the carrier label, and then back through **RMA**, warranty scope and disposition.
- **The same code → commerce,** into Merchant Center, Shopping, social catalogs, and the conversion record.

GS1 already has the syntax for the part most teams hand-roll. A Digital Link carries more than the product:

```
/01/<gtin>/10/<batch>/21/<serial>
```

```
01 = product      10 = batch / lot      21 = serial
```

That is how you get from *this product* to *this unit* — which is the difference between "we ship a device with that vulnerability" and "these 4,200 units, in these shipments, to these customers."

One thread: factory → warehouse → sale → return → recall. It exists whether or not anyone models it. The question is only whether it is one identifier or four reconciliations.

A return is the hardest test in the estate, because it exercises every per-market fact simultaneously — and it does so *after* the margin is gone.

A US return is one reverse lane, one tax treatment, one policy you wrote yourself. Cross-border, each of those splinters:

- **Withdrawal is a right, not a policy.** The EU's 14-day withdrawal right is granted to the buyer, and the legal guarantee runs two years, regardless of what your terms page says.
- **Customs runs backwards.** Duty and import VAT already paid need relief or refund; the unit has to import into wherever it is going. Wrong paperwork and the return costs more than the item.
- **Merchant of record decides who may accept it.** If the seller in that market is a separate entity or a marketplace, the return is theirs.
- **Disposition is regulatory, not logistical.** A unit returned in the EU may not be resellable in the US and vice versa — different marks, different radio approvals, different firmware locale. "Restock" is a compliance decision.
- **Refund rail and currency** must match the original capture.

Every one of those is a **per-market fact**. And most systems model market as a *display preference* — a currency symbol, a language, perhaps a tax rate. So RMA logic ends up hard-coded per country, in whichever system was nearest, by whoever was on that project. That is the spiral: not complexity, but the same decision re-implemented in six places with no shared key.

The fix is not to write more RMA logic. It is to stop reshaping the data at each border. Market becomes a first-class record — currency, tax treatment, locale, consent regime, merchant of record, withdrawal terms, disposition rules — and the identifier resolves against it. One thread, annotated by market, rather than one thread per market.

Those operational numbers used to live in a world with a **reconciliation window**. The warehouse and commerce could disagree until month-end and someone fixed it in a spreadsheet. Nobody was harmed, because nobody asked at event time.

Shopify and Google closed that window. Both moved the trust boundary server-side — Consent Mode v2, Merchant Center matching, the Shopping Graph entity. Commerce now resolves *at the event*. The warehouse plane still resolves *nightly*. A divergence that used to be a month-end chore is now a live inconsistency between what is advertised, what is on the shelf, and what the firmware reports.

Then the obligations land on exactly that seam:

- **CRA Article 14**, from 11 September 2026, obliges reporting of actively exploited vulnerabilities — including for products already on the market. Scoping that report requires firmware ↔ serial ↔ GTIN ↔ shipment ↔ customer. Break one link and you cannot answer.
- **The EU Omnibus price rules** require a 30-day prior-lowest reference, which must attach to the same entity the ads bid on, or your evidence is about a different product than your promotion.
- **Returns and warranty** arrive as a physical code and must resolve to terms, batch and firmware.

It is worth separating this from the AI conversation, because it was true before and stays true regardless.

Whatever mix of channels you sell through — own store, marketplace, retail partner, social — Google is where the product gets **matched to an entity**. The

Merchant Center feed resolves your item into the Shopping Graph; Shopping and Performance Max bid against that matched entity; Meta, TikTok and Pinterest catalogs consume the same identifiers; conversions attribute back to it; and AI answer surfaces read the same graph. Adding AI raised the stakes. It did not create the dependency.

Which means the identifier is not only an internal join key. **It is the thing that decides whether your product exists as one entity in the layer most discovery passes through, or as an orphan.** A missing or wrong GTIN does not produce a clear error — it produces poor matching, and poor matching looks like a bidding problem for months before anyone suspects the data.

Detection that runs in real time, not at month-end

The useful consequence is that this loop can be closed automatically, and most teams do not close it.

- **Merchant Center product status** is queryable per item — disapprovals, warnings, identifier mismatches, price and availability discrepancies against the landing page. That is an API surface, not a dashboard you have to remember to open.
- **The Merchant Center BigQuery transfer** lands product status and performance into your warehouse on a schedule, joinable to your own catalog.
- **GA4's BigQuery export** delivers event-level data with the campaign and item parameters intact.

Put those beside the resolver mapping and you have a genuine detection loop: **the identifier the warehouse holds, the identifier the feed submitted, and the identifier Google matched — compared continuously, with a difference raising an alert rather than waiting for a quarterly audit.**

That is the same instrument-then-automate order as everywhere else in this document. The comparison is worth building even if no agent ever acts on it, because it answers a question nobody can currently answer: *are the physical estate, the commerce estate and the discovery estate talking about the same product right now?*

And once it exists, it is exactly the signal an agent can act on — reconciling a mismatch, re-submitting a corrected item, flagging a batch whose disposition changed. But the detection has to exist first. A loop pointed at an environment where nobody knows which identifier is authoritative will iterate confidently on the wrong one.

THE BEHEMOTH, AND WHY YOU SHOULD NOT TOUCH IT

Most of this data currently lives behind a planning estate — JDA/Blue Yonder, Manhattan, SAP retail — plus GDSN data pools, supplier onboarding by flat file, and EDI still carrying the volume. Long implementations, change control rather than editing, batch by design, and an item master that quietly became the de facto product truth because the supply chain trusts it more than commerce does. Adding a field is a project.

Do not migrate it. Map it.

You do not need the planning system to hold the GTIN. You need a mapping you own — planning item ↔ GTIN ↔ commerce variant ↔ firmware revision — resolved server-side at event time, with its own record. The planning estate keeps doing exactly what it does. The resolver absorbs the impedance mismatch, and the batch cadence stops being a correctness problem because the mapping is authoritative even when the nightly run is not.

That is a table and a function, not a re-platforming. It is also the only version a supply-chain owner will agree to, because it asks nothing of them.

GS1 Digital Link changes the identifier from *a number you send to a data pool* into a **URI you host**:

```
https://yourdomain.com/01/09506000134352
```

The GTIN is still GS1's. **The endpoint is yours.** What that URL returns — price history, SBOM, consent notice, verification certificate, localised content per market, return instructions for the market the scan came from — is your data model, on your infrastructure, changeable without a data-pool sync or anyone's certification cycle.

That is the first genuinely web-native thing in this stack, and it routes around the flat-file world without confronting it. The scanner still gets a number. Everything else stops being a CSV.

WHO GETS TO CHANGE IT

The last piece is governance, and it decides whether any of this survives contact with an organisation.

If a business analyst needs to add a country code — one field that propagates to WMS, ERP, RMA and customer service, and the location-based funnel — that change should be made by the person who **owns the revenue outcome in that market**, under a bounded mandate: scoped, capped, expiring, revocable, and recorded. Not raised as a ticket to a team ten timezones away with a change window.

The usual objection is governance: *who is accountable, where is change control*. It deserves a straight answer, because in the old architecture the objection was correct — unbounded access with no record genuinely was reckless, and the queue was the only control available.

But a queue was never actually a control. **Whoever picks up a ticket holds full access, and the only limit is what they chose to type.** A mandate is scoped, capped, expiring, revocable, and records what was refused as well as what was done. The objection is right about the risk and wrong about which option carries it.

WHAT TO DO BEFORE YOU HAVE GTINS

You are not blocked. In order:

1. **Start the GS1 licence.** It is the long pole and it is commercial.
2. **Make market a first-class record** — not a display preference. This is the work that pays off regardless of Sunrise.
3. **Bind every identifier you already have** — MPN, SKU, planning item — to a single product entity now, so the join is correct on the day GTINs arrive rather than being retrofitted.
4. **Stand up the resolver** at `/01/...`, even returning a stub. The route existing is what lets everything else be tested.
5. **Instrument the thread** — factory, warehouse, sale, return — so you can answer *which units* before anyone asks.
6. **Artwork last.** It is the only step that cannot be changed after it ships.

HONEST DISCLOSURE

At the time of writing, the catalog behind this document carries `mpn` and `sku` on every product and **no GTINs at all**, and the `/01/...` resolver is not built. The QR pipeline exists and every minted code carries its product identifier; the GTIN does not exist yet, and a Digital Link cannot be minted without one.

That is the ordinary starting position, and it is stated here because a readiness article written by someone pretending to be finished is not worth reading.

THE ENCOURAGING PART, WHICH IS EASY TO MISS

None of this requires the legacy estate to change, and that is worth ending on.

Commerce was always server-side. Catalogs, orders, inventory, pricing — all of it resolved on a server long before anyone said "server-side" as a strategy. The past decade of client-side tags and browser-resident tracking was the aberration, not the baseline. What has happened recently is a return: Google moved its consent and measurement boundary back to the server, and **Shopify joined it**, resolving identity, consent and catalog at the event rather than in the page.

Which means there is now a **real-time plane above the batch estate** — and you do not have to earn your way onto it by first fixing everything underneath.

That is why LLM-based and agentic commerce is achievable in an enterprise that still runs planning on a nightly cycle and still receives a supplier CSV on Tuesdays. The agent does not query the planning system. It calls a bounded server-side function, which resolves against a mapping you own, which was *fed* by the CSV but is not *arbitrated* by it. The flat file becomes a source among sources rather than the thing that decides what is true.

Stated plainly:

- **The CSV stays.** It is a delivery mechanism, and there is nothing wrong with a delivery mechanism.
- **The planning estate stays.** It keeps planning. Nobody has to justify a migration.
- **The resolver is the join**, holding the identifier mapping and answering at event time, with a record.

- **Agents talk to the resolver**, never to the legacy directly, so their authority is bounded and their actions are attributable.

The batch cadence stops being a correctness problem the moment something authoritative sits above it. That is the whole trick, and it is a smaller piece of work than the size of the surrounding systems suggests.

Sunrise 2027 is a retailer capability date. The work is a data-model change. And the identifier you assign this year decides whether a scan in 2028 resolves to one product — or to nothing.
